Supplementary Material 1 for:
Continuous curation of an *Alteromonas macleodii* metabolic model

Helen Scott, Devlin Moyer, Osnat Weissberg, Franziska Kratzl, Waseem Bashir
Valiya Kalladi, Rogier Braakman, Mary Ann Moran, Daniel Sher, Daniel Segrè

Contents

1 Motivation **2**
1.1 Why do we need this? . . . 2
1.2 What have people been doing before this? . . . 2
1.2.1 MEMOTE . . . 2
1.2.2 standard-GEM . . . 2
1.2.3 human-GEM and yeast-GEM . . . 3
1.3 What's new here . . . 3
1.4 What's the cost? . . . 3

2 Background/Primer **3**
2.1 What is GitHub? . . . 3
2.1.1 Diffs . . . 3
2.2 Branches . . . 5
2.3 Unit Tests . . . 5
2.3.1 What makes a Good Unit Test? . . . 6
2.4 Automation with GitHub Actions . . . 7
2.4.1 Defining an Action with a YML file . . . 7

3 Set-Up **8**
3.1 What's in a Name?: Choosing your Model and Repository Name . . . 8
3.2 Branching Strategy . . . 9
3.3 Repository Structure . . . 9
3.3.1 `.github/` . . . 10
3.3.2 `code/` . . . 10
3.3.3 `data/` . . . 10
3.3.4 `model/` . . . 11

4 The Continuous Curation Loop **12**
4.1 Step 1) Curate . . . 12
4.1.1 Removing Reactions/Metabolites . . . 12
4.1.2 Commits . . . 13
4.1.3 Pull Requests . . . 13
4.2 Step 2) Test . . . 14
4.2.1 GitHub Action . . . 14
4.3 Step 3) Report . . . 15
4.3.1 Examples of Scripts Used for Model Curation . . . 15
4.4 Step 4) Release . . . 16
4.5 Step 5) Run . . . 17
4.6 Step 6) Monitor . . . 18

5 Why can't you just make it for me? **19**

6 Develop Good Code/GitHub Habits **19**

7 Glossary **19**

References **21**

GEMs are, at their heart, a software product, and we took lessons from software development and applied them to the model curation process. We term this constant testing and iterative model improvement strategy "continuous curation", inspired by continuous integration (CI) for traditional software. This included tracking all changes using version control, having multiple curators collaborate and propose changes by working on branches and opening pull requests, testing the model with defined pass/fail tests, and automatically generating artifacts for curator inspection.

- None of this is really new
  - we borrowed all of this from software engineering
  - other groups do or are developing similar things
    - we took some things directly from human-GEM and standard-GEM
  - the field is crystallizing
  - but the field lacks a single reference point for guidelines

1 Motivation

1.1 Why do we need this?

- GEMs are important tools
  - But their quality is often questioned
- Manual curation is hard/messy
  - Manual curation can be an overwhelming task- the typical advice of just go "pathway by pathway" can be paralyzingly large
- Manual curation can last for a long time (including decades, spanning many people and projects)
  - Often one group publishes a model, then another may find it, modify it, and publish a new paper, resulting in branching sets of models

1.2 What have people been doing before this?

1.2.1 MEMOTE

- MEMOTE exists (1)
  - but it's more of a benchmarking tool- a lot of things is just about the file
    - test what the custom tests ever did
- They had a TRAVIS integration where it would run, but MEMOTE can never fail, it always required a human to read, understand, and evaluate the tests

1.2.2 standard-GEM

- standard-GEM is a set of requirements for the format of GEMs versioned with git (2)
- no tests
  - they show running MACAW in a workflow, and they call it `macawTests.py` but it's not a test, it doesn't assert anything, equivalent to a "script"
  - code/test and MEMOTE are only possibility/capability, not a requirement or even a recommendation
- technically just because it's on git doesn't mean you're regularly pushing- you could upload everything once and meet standard-GEM requirements, even if not in spirit

1.2.3 human-GEM and yeast-GEM

- Human-GEM
  - 2023: check-metabolictasks.yml
    - "metabolic tasks" on push and PR
    - using a self-hosted MATLAB runner
  - 2024: Gene essentiality workflow
  - 2026: model-qc.yml
- yeast-GEM
  - 2018: automated benchmarking MEMOTE + Travis CI
  - 2020: MEMOTE on GitHub actions + YAML-vaildation
  - 2026: model-qc.yml
- A lot of things were not enforced, just reminders and check boxes

1.3 What's new here

- Our specific unit tests- not really clear who go to running it as CI first...
- Biomass component producibility heatmaps
- Improved tracking removed reactions/metabolites

1.4 What's the cost?

- Requires set up and maintenance
- Requires learning more about Git and automation than someone with a biology background may currently have

2 Background/Primer

2.1 What is GitHub?

Version control is critical for model curation because it tracks the "who", "what", and "when" of all changes made to the model. Who edited the model file, when did they make the edits, and what exactly was changed. It also maintains the historical versions of the model file, so at any time you can revert changes and return to an older version of the model.

Version control keeps a historical record of changes made to tracked files in a specialized database called a repository (or "repo"). Git is the software tool that enables version control, and GitHub is one popular cloud-based platform to host Git repositories, that also offers other functionalities such as issue tracking and wiki hosting. While we used GitHub, and will use GitHub-focused terminology (e.g., pull requests, actions) it not the only option for hosting Git repositories, other popular options include GitLab, Bitbucket and Azure DevOps, each of which have analogous tools to those we describe here and could similarly be used for a continuous curation pipeline.

- A Git repository lives in two places...
- Local vs Remote, basic terminology (commit, push, pull, etc.)

2.1.1 Diffs

- How to read a diff

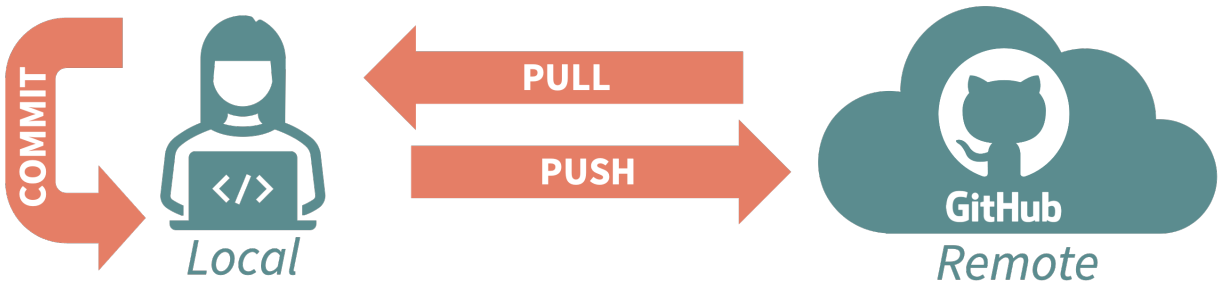
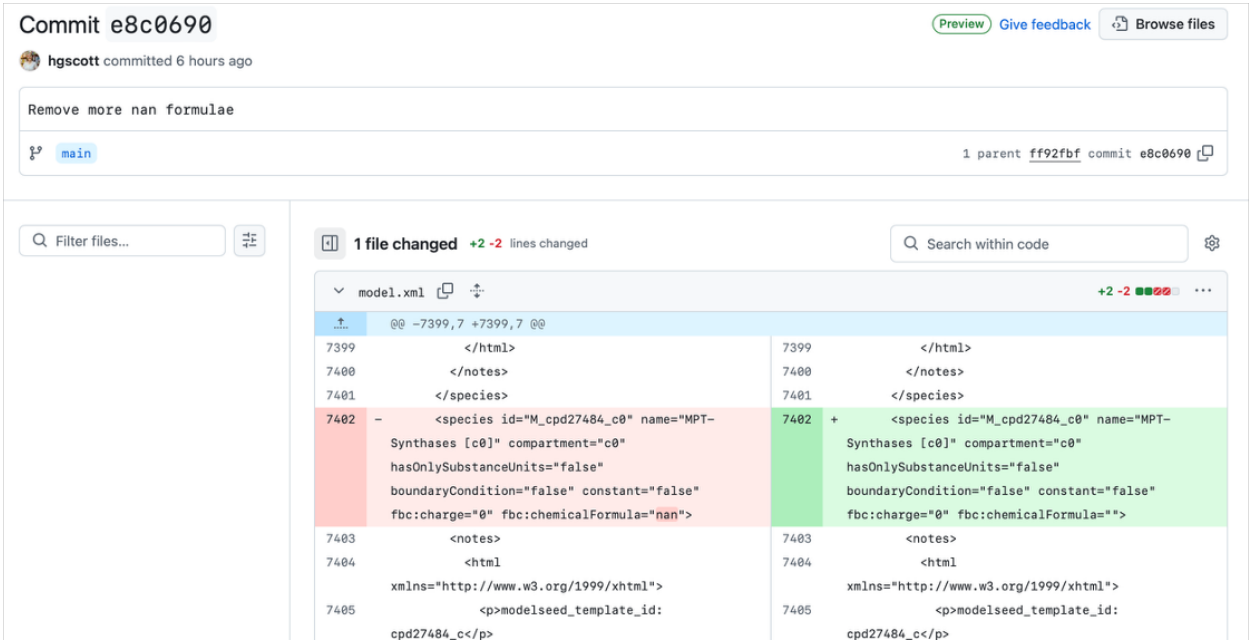
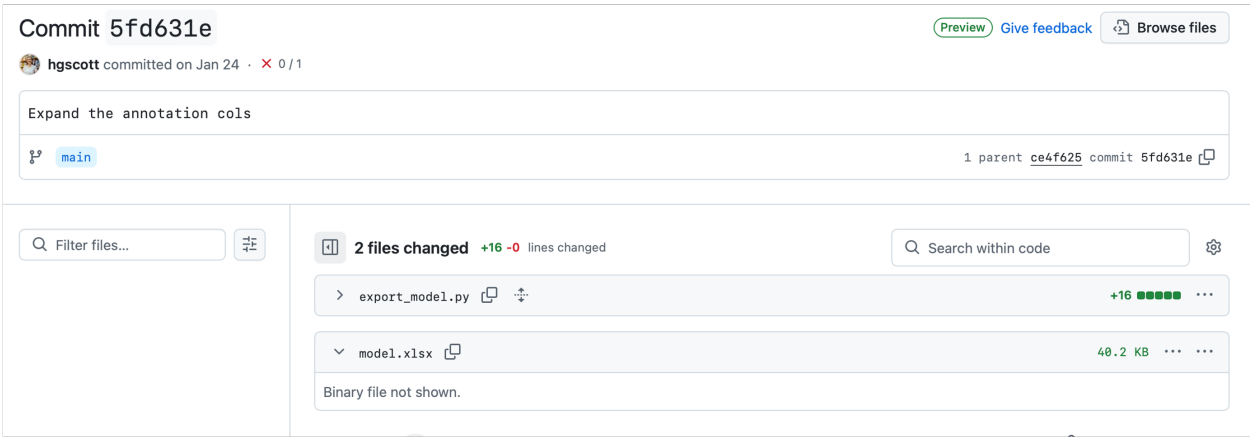


Figure 1: An introduction to basic Git terminology: Local and remote repositories, commit, push, and pull

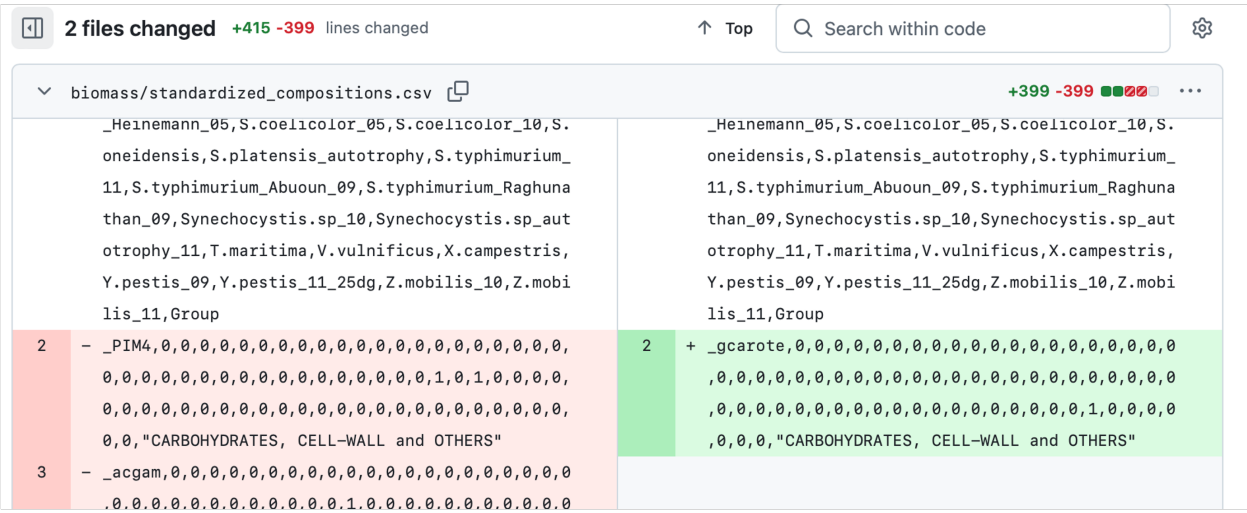


- Not all diffs are equally helpful
  - File types to avoid
    - Microsoft office files: e.g., xlsx

Excel



CSV


2 files changed +415 -399 lines changed
Top
Search within code
biomass/standardized_compositions.csv +399 -399
_Heinemann_05,S.coelicolor_05,S.coelicolor_10,S.oneidensis,S.platensis_autotrophy,S.typhimurium_11,S.typhimurium_Abuoun_09,S.typhimurium_Raghunathan_09,Synechocystis.sp_10,Synechocystis.sp_autotrophy_11,T.maritima,V.vulnificus,X.campestris,Y.pestis_09,Y.pestis_11_25dg,Z.mobilis_10,Z.mobilis_11,Group
2 - _PIM4,0,0,0

- Imagine you have a python module called `hello` with a single function, also called `hello`, that says "Hello" to a person, given their name:

```
def hello(name: str):
    message = 'Hello, ' + name + '!'

    return message
```

- To write the unit test, you would make a new file, by convention in a directory named "test" or "tests", and name the file "test_[module name]", for that individual python module, so in our case "test_hello"
- While there are other testing frameworks, we show the python standard-library testing framework, unittest
- In your test, you first import unittest, and your function that you are testing

```
import unittest

from hello import hello
```

- All tests are put within a class (for unittest)

```
class TestHello(unittest.TestCase):
```

- Within that class you can write multiple tests
- For example, first test an expected input, e.g., "Helen"
    -
```
def test_helen(self):
    # Define a name
    name = 'Helen'
    # Call the function
    message = hello(name)

    # Assert that the function returns the correct string
    self.assertEqual(message, 'Hello, Helen!')
```
    - You define if the output is correct or not with an assert statement
    - There are other options for assert statements that make sense for different kinds of tests
        - Table/list of assert statements
- Then test an edge case, something that could return a bad output if your function is not properly written, e.g. a number

```
# Add a test checking that the function fails when an integer is passed
def test_integer(self):
    # Define a name
    name = 123
    # Call the function
    with self.assertRaises(TypeError):
        message = hello(name)
```

- At the end of the file- set it to run all of the classes

```
if __name__ == '__main__':
    unittest.main()
```

- Unit tests can be run manually- e.g., a developer runs them on their own computer and verifies that they all pass before pushing code
- Or they can be run automatically (e.g., as part of a GitHub action)

2.3.1 What makes a Good Unit Test?

- CITE (3) and (4)
- Needs to pass/fail, have an expected outcome, not generate an artifact

    - Boolean result- pass or fail
    - Self-validating — it returns pass or fail, not output a human must inspect
- One reason to fail — a test asserting five things tells you "something broke," not what
- Fast — slow tests don't get run, and a suite people skip provides no safety
- Independent — no test depends on another's state or on run order
- Repeatable — same result on any machine, any time; no network, no clock, no randomness
- Deterministic — the intermittent test is worse than no test, because it trains people to re-run until green
- No logic in the test — conditionals and loops inside a test can themselves be buggy, and then you're debugging your test
- Tests behavior, not implementation — a test that breaks when you refactor without changing behavior is a liability
- A name that says what broke without opening the file
- Table with columns:
    - Example of good unit tests
        - Test that a script generate the correct data and saves a plot with the correct path
    - Example of bad unit tests
        - Generate a figure, that you need to look at
        - A test you know will fail, and you will just ignore it (skip the test or mark it a known failure instead)

2.4 Automation with GitHub Actions

We used GitHub Actions to automate running tests, generating reports, and releasing the model. GitHub Actions is tool built in to GitHub that can run pre-defined workflows on GitHub servers. It is typically used in software development for Continuous Integration testing and for automated deployment.

2.4.1 Defining an Action with a YML file

- The steps of an Action are defined in a workflow file
- Workflow files use YAML syntax, and the files must end in `.yml` or `.yaml` and must be stored in `.github/workflows` in the repo
- For more information on YAML see: Learn YAML in Y Minutes and the GitHub Workflows and Actions documentation
- Our workflow files have the following key sections, others exist, see more exhaustive documentation:
    - `name`
    - `on`
    - `permissions`
    - `concurrency`
    - `jobs`
- For an example we will walk through what a YAML file for automated running of the tests for our `hello` module.
- We first define the name of the workflow, this will show up in the Actions tab on GitHub
- In the YAML file we write:

```yaml
name: Test-Hello
```

- Then we define when this workflow will run, what will trigger it to run:

```yaml
# Run this every time anything is pushed to the repository
on: push
```

This means that whenever changes are pushed to the repo, this workflow will run.

Other possible triggers for a workflow are (again, not exhaustive): * `push`: when changes are pushed to any branch or tag * `schedule`: on a specific schedule, i.e., every day * `workflow_dispatch`: manually triggered with a button

The next section in the workflow file, `jobs` defines the actual work done in the workflow. * A workflow run is made up of one or more jobs that can run sequentially or in parallel. * In this YAML file we will only have one job, `test`

```yaml
jobs:
  test:
```

- Within test, we first define what virtual machine this workflow is running on
  - This means what computer environment
  - Note this environment starts totally clean every time, so you can always test what it would be like to start on a totally new computer, this is good because you know exactly what needs to be done/installed (you never have a "well it runs on my machine" moment), but it does mean you need to set it up with every run

```yaml
  test:
    # The machine it runs on
    runs-on: ubuntu-22.04
```

- The job is defined by a series of `steps`, a sequence of tasks that will be executed as part of the job.
- Each step can be named with `name`, and either calls a published action with `uses` or executes shell commands in the runner with `run`.
- Since we are starting in a totally blank virtual machine, the first few steps are usually dedicated to setting it up:

```yaml
  steps:
    # Copy the repository onto that machine
    - uses: actions/checkout@v7

    # Put a known version of Python on it
    - uses: actions/setup-python@v6
      with:
        python-version: "3.11"

    # Install what the tests need
    - name: Install dependencies
      run: pip install pytest
```

- Here we checked out the repository, so our virtual machine has all of the code on our repo, then we installed a specific version of Python (using a specific version is good, because it won't drift as Python is updated, so things won't break). Then we can install our specific dependencies, in this case `pytest`
- The next step is to actually run the tests:

```yaml
    # Run them
    - name: Run tests
      run: pytest
```

Then you get a clean read out if your test passed or failed with every change, you never need to run the tests yourself, or count on other people running them.

3 Set-Up

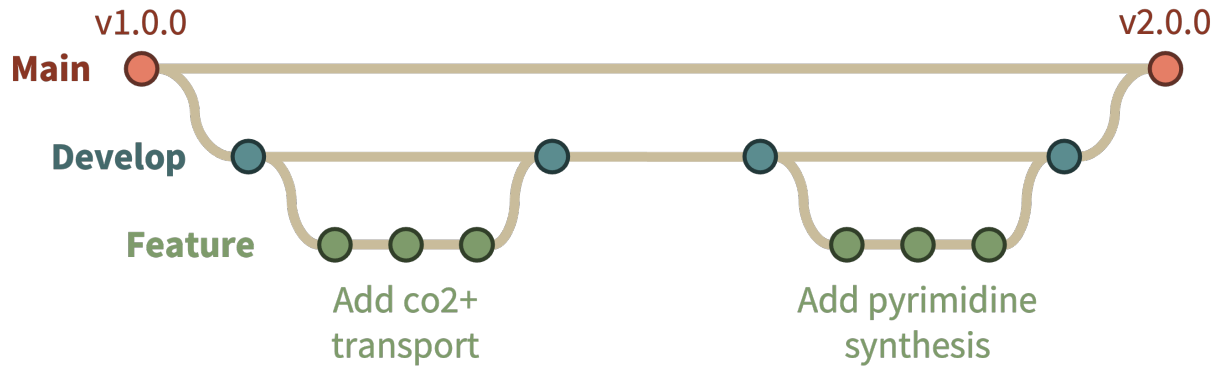
3.1 What's in a Name?: Choosing your Model and Repository Name

- BiGG names models following a pattern, of i{Author's Initials}{Number of genes in the model} (5)

    - the i in the name refers to an in silico model (that is, a computer model)
    - 'i' is followed by the initials (XX) of the person who developed the model and then the number of genes (xxx) included in the model. (5)
    - e.g.,
        - iJR904 is an in silico model (i), built by Jennifer Reed (JR), containing 904 genes (904)
        - the current E. coli model is iML1515
        - presumably, M stands for Jonathan Monk, L stands for Colton Lloyd the two first authors of the related paper
    - iJO1366 is by Jeffrey Orth
- But this naming is inherently a snapshot, and would have to be continuously updated as the model evolved
    - Every time you add or remove a gene you would need to change the number
    - As new curators contribute to the model or take over, the author list may expand or change, or people may be annoyed to leave an old authors name at the prime spot
- It would be better instead to have a single name that is not dependent on the author or gene list, that is specific to the organism
    - It would be nice to instead have on model with different versions
    - e.g. e.coli model v1, e.coli model v2, to make it clear the lineage of the model
- The recommendation from standard-GEM is to name the repository and model {something}-GEM, a common name, KEGG organism, or taxonomy-derived short name
    - e.g. Human-GEM and yeast-GEM
    - We followed that advice, and named the model MIT1002-GEM

3.2 Branching Strategy

We chose to use a branching strategy based on the popular GitFlow strategy (6). We had two long-lived branches, `main`, the main branch, which had the official releases of the model, and `develop`, the development branch, where all accepted changes to the model were integrated before an official release. All changes made the model were made on feature branches that branched off of and were merged back into `develop`. This ensured that any new feature development did not disturb the main model.



3.3 Repository Structure

Following and building upon the Standard-GEM (2) specification, the MIT1002-GEM repo is structured as:

```
MIT1002-GEM/
|-- .github/
|   |-- ISSUE_TEMPLATE/
|   |-- workflows/                  # CI, release, standard-GEM update
|   |-- CONTRIBUTING.md
|   `-- pull_request_template.md
```

```
|-- code/
|   |-- test/                       # assert pass/fail; block a merge
|   |-- scripts/                    # generate artifacts for a human to read
|   |-- tools/                      # shared importable functions
|   `-- ...                         # exploratory analyses (biomass, pangenome)
|-- data/
|   |-- deprecated_identifiers/     # what was removed, and why
|   |-- known_growth_phenotypes.tsv
|   `-- README.md
|-- docs/
|   `-- continuous-curation.md
|-- model/
|   `-- MIT1002-GEM.xml             # the only model file on develop
|-- CHANGELOG.md
|-- README.md
|-- requirements.txt
`-- version.txt
```

3.3.1 `.github/`

- The `.github/` directory is to hold all GitHub-specific configuration files (e.g., the contributing guidelines, code of conduct, issue templates)
- the `workflows` subdirectory within it, holds the YAML workflow files
  - In MIT1002-GEM we have 5 workflow files within `.github/workflows`, the specific workflows will be described in detail in the following section

3.3.2 `code/`

- Standard-GEM requires a `code` directory to house all the code used in generating the model. We further refine `code` into 4 categories
  - Tests
  - Scripts
  - Tools
  - Exploratory Analyses
- Tests are all unit tests (as described above)
- Scripts differ from tests because they cannot "pass" or "fail", and instead generate artifacts (e.g., plots) for a human curator to look at
- If you can state in advance what is right or wrong- make a test
- If you can use the information, but you're not sure what is right- make a scripts
- Tests can block a merge- you need to pass all tests before you can merge a PR
- Scripts don't block anything
- Automated- also ran through GitHub actions
  - Doesn't include any arguments or hand editing
- Should be something you need continually- not just a one off for a specific question
  - If no one is looking at the artifact- you don't need the script
  - A one-off for a specific question is an exploratory analysis and can live in its own directory, e.g., `biomass` or `blast`
- They might share code with tests
  - E.g. the growth phenotype plot uses the same code as the growth phenotype ratchet test
  - Make sure shared code lives in tools, not recreated in both files

3.3.3 `data/`

- This folder contains the data used in generating the model.
- It must also include a README.md file that describes how the folder is organized.

3.3.4 `model/`

- This directory houses your model file(s)
- The model file should have the same name as your repository ({something}-GEM)
- There are many different file formats that you can use (e.g., JSON, MATLAB, SBML, etc)
- Different tools use different file types/different users may have different
- We strongly recommend that you pick one file type to be your "default" and have only that file on your `develop` branch, and export the model to all other file types only on the `main` branch

3.3.4.1 What file type to use as your default?

- the eXtensible Markup Language (XML) is a markup language and file format for storing, transmitting, and reconstructing data.
- "The Systems Biology Markup Language (SBML) is a file format for representing computational models in a declarative form that different software systems can exchange" (7) and is encoded in XML
- SBML is formally defined in the specification documents
  - The major advantage is that SBML files can be validated against the published SBML specification
  - This only constrains form, not the biology, just because the SBML is valid, doesn't mean the biology within it is correct
- SBML looks intimidating, but it's straight forward
  - It's plain text, so you can open it and read the raw file yourself (i.e., it's not binary), which also means it diffs cleanly
  - Most of the time, you don't need to read an SBML file, you work with the model as an object loaded into a tool (e.g., COBRApy)
  - But sometimes you will need to look at the raw file, e.g., when you are looking at a diff
  - The file is big, but very repetitive
  - XML mechanics
    - Tags open and close
      - `<species> ... </species>`
      - A tag can close itself: `<compartment id="c0" constant="true"/>`
        - the `/>` means no children
    - Attributes live inside the opening tag
      - Like writing on the outside of the box
      - Can only be one simple value, and can't contain anything
      - What this looks like in the file

```
<species id="M_cpd00443_c0" name="4-aminobenzoate" fbc:charge="-1"/>
         |-------- all three of these are attributes --------|
```

    - Children live between the tags
      - Boxes inside of the box
      - Can be multiple of something, or something with parts of its own
      - What this looks like in the file

```
<reaction id="R_octanoyl_xfer" reversible="false">   <-- attributes
  <listOfReactants>                                  <-- a child element
    <speciesReference species="M_cpd11470_c0"/>      <-- a child of that child
  </listOfReactants>
</reaction>
```

    - indentation shows nesting
- Elements relevant to GEMs

Element	Holds
`<listOfUnitDefinitions>`	units
`<listOfCompartments>`	c0, e0
`<listOfSpecies>`	metabolites
`<listOfParameters>`	the flux-bound values

Element	Holds
`<listOfReactions>`	reactions
`<fbc:listOfObjectives>`	what FBA maximizes
`<fbc:listOfGeneProducts>`	genes

- some tags say `fbc:` because they are specifically from the Flux Balance Constraints package which adds the necessary attributes for FBA (8)
- Nothing really contains anything- they only point at each other
  - E.g., a reaction doesn't contain a metabolite, it points at the metabolite using its ID
- Example
  - Reaction

```
<reaction id="R_octanoyl_xfer" name="Octanoyl transferase" reversible="false"
    fbc:lowerFluxBound="cobra_0_bound" fbc:upperFluxBound="cobra_default_ub">
<listOfReactants>
    <speciesReference species="M_cpd11470_c0" stoichiometry="1" constant="true"/>
</listOfReactants>
<listOfProducts>
    <speciesReference species="M_cpd11493_c0" stoichiometry="1" constant="true"/>
    <speciesReference species="M_cpd14953_c0" stoichiometry="1" constant="true"/>
</listOfProducts>
<fbc:geneProductAssociation>
    <fbc:geneProductRef fbc:geneProduct="G_WP_049588698__46__1"/>
</fbc:geneProductAssociation>
</reaction>
```

  - Species it points to: `xml <species id="M_cpd00443_c0" name="4-aminobenzoate" compartment="c0" fbc:charge="-1" fbc:chemicalFormula="C7H6NO2" .../>`
- A couple random points of clarification
  - COBRApy adds prefixes to all the IDs because SBML IDs must begun with a letter or underscore, and strips them when actually displaying the file
    - `R_` for reactions
    - `M_` for metabolites
    - `G_` for genes
  - `__46__` is an escaped period
    - SBML IDs can't contain dots
    - So COBRApy encodes them as `__` plus the ASCI code (46=.)
    - So `G_WP_049588698__46__1` is really `WP_049588698.1`.

4 The Continuous Curation Loop

The Continuous Curation loop consists of 6 steps: 1. Curate 2. Test 3. Report 4. Release 5. Run 6. Monitor

4.1 Step 1) Curate

NOTE: We do not discuss here how to make curation decisions, but rather how to implement them

- How big is one curation task?

4.1.1 Removing Reactions/Metabolites

- Palsson said to do it (9)
- What we took from human-GEM: the table
- What is new
  - The standard vocabulary of reason

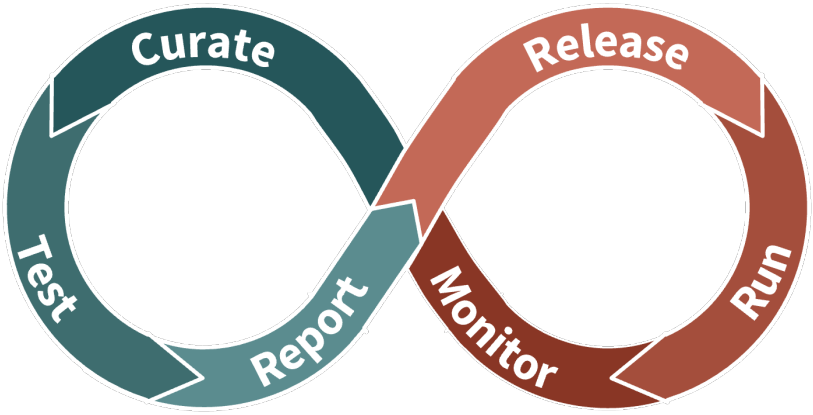


Figure 2: Continuous Curation is an iterative process with the following steps: Curate, Test, Report, Release, Run, and Monitor.

- The list of removed things in the model file itself
  - And the test to make sure it does not drift
- The test that no old reactions are still in the model file

4.1.2 Commits

- Commit often- don't just wait until the end of all your changes
- Use concise descriptive commit messages
- Use semantic commits
  - [optional scope]:
  - We got this from standard-GEM
  - type
    - chore: updating toolbox, data files, etc.
    - doc: updating documentation or explanatory comments in functions.
    - feat: new feature added, e.g. new reaction / metabolite / function / etc.
    - fix: something that was incorrect in the model and now has been corrected.
    - refactor: see code refactoring.
    - style: minor format changes of model, functions or data (spaces, semi-colons, etc., no code change)
  - scope
    - optional
    - refers to the receiver of the action, i.e. what part of the model/data are you modifying
  - description
    - concise description of what you did
  - some examples (direct copy from yeat-GEM contributing guidelines) |commit|commit message| |:—:|:—:| |Add new rxns|`feat-rxn: methanol pathway`| |Remove a metabolite|`fix-met: duplicated citrate`| |Add metabolite formula|`feat-met.prop: carbohydrate formulas`| |Fix rxn stoichiometry|`fix-rxn.prop: complex V stoich coeffs`| |Update gene IDs|`fix-gene.annot: update IDs from SWISSPROT`| |Format name of compartment|`style-comp.annot: remove uppercases`| |Split a rxn in 2|`refactor-rxn: split isomerase in 2 steps`| |Add some data|`feat-data: metabolomics data`| |Update documentation of function|`doc: addDBnewRxn.m`| |Update toolbox|`chore: update RAVEN version`|

4.1.3 Pull Requests

One critical component of the history of changes to the model is the "why"- why was a change to the model made (e.g., was a reaction found to have genomic evidence, was there a mistake in the biochemistry database,

etc.). There are text fields in the model file itself where this information can be stored, and there have been cases in the past of defined "codes" used to represent different types of evidence that support each reaction (9) (they used numeric codes 4 = direct biochemical evidence, 3 = genetic evidence, 2 = sequence/homology evidence, 1 = physiological evidence, 0 = no evidence (included only because it improves modeling)) however we have found that these are not well used, lack standardization across the community, and are often not comprehensive enough to fully explain the reasoning behind each change. We instead elected to document these in issues and pull requests on the repository. Issues can be used as a sort of electronic lab notebook. To ensure that all curators (present and future) are reminded to document their reasoning, a pull request template was used.

- Open a pull request, that starts the cycle

4.2 Step 2) Test

- Testing code is important, testing the model is just as important
- Typical software tools can be used, but some concepts need to generalized
- We can differentiate the tests used in MIT1002-GEM by what they tested, the model, the data files, the helper tools, or consistency between two things. A single test file can contain multiple different kinds of tests, so some file names may repeat across the list. #### Examples of Unit Tests on the Model
- In traditional software engineering, the unit being tested is often a function, however for the case of model curation, we are testing the model as a whole, but can write tests to focus on individual aspects of the model
  - Why testing a model is not like testing a function (10)
- The ones we present here are by no means an exhaustive list of everything that could or should be tested.
- Many of these tests use previously published tools (e.g., MEMOTE), but we found that by implementing them with unit tests on a GitHub action it was easier to track model performance over time and recognize errors introduced into the model quickly.
- Part of the MIT1002-GEM repo is helper functions (i.e. for XXX), these functions, just like any other functions in a python module should be tested.

4.2.1 GitHub Action

- In MIT1002-GEM we run them all automatically in the CI-workflow `Test-and-Report`, finding and executing tests with `pytest`
- The next line in the file, defines when the workflow will run. `Test-and-Report` says:

```
on:
  pull_request:
```

so that it runs whenever a pull request is opened or changes are pushed to the PR branch * Within the `test` block, `steps` represents a sequence of tasks that will be executed as part of the job. The steps of the `test` job are to: Check out the repository, so the job can access your files, install all required dependencies, and then run the tests. * ubuntu-22.04 is a specific version of the Linux runner, using a specific version is good because it will not change whenever the latest Linux runner is updated, so it is less fragile.

```
jobs:
  test:
    runs-on: ubuntu-22.04
    steps:
      # Checks-out your repository under $GITHUB_WORKSPACE, so your job can access it
      - uses: actions/checkout@v3
        with:
          ref: ${{ github.head_ref || github.ref_name }}
          fetch-depth: 0
```

```
    # Install everything I need
    - name: Install Dependencies
      run: |
        python -m pip install --upgrade pip
        python -m pip install -r requirements.txt

    # Run the custom tests
    - name: Run Custom Tests with pytest
      run: pytest
```

4.3 Step 3) Report

4.3.1 Examples of Scripts Used for Model Curation

- Using the same underlying code as the growth test, we generated a plot of which experimentally known growth phenotypes the model matched or not. The heatmap visualization was useful for sharing results.
- While this graph is useful to grasp model performance at a glance, it was not the most instructive when gap-filling the model (just knowing that the model does not grow does not help you find a gap). Instead we checked the model's ability to produce each individual biomass component (i.e., we added a demand/sink reaction for each biomass component that take the metabolite and removes it from the system (similar to an exchange reaction), and looped through the list of biomass components and set each as the objective, maximizing the flux through that sink reaction. A positive value indicated that the model was capable of producing that biomass component. This helped narrow down searches for gaps (e.g., could say that a subset of amino acids was not producible, therefore there must be a gap in that pathway). When using an objective other than the biomass, we considered if there should be free transport/exchange/sinks for all biomass components simultaneously or if only the one being maximized should have a sink. Theoretically, there could be components whose production is tied and without flux through the biomass reaction, dead ends could appear that block flux. #### GitHub Action Workflow The following job, `report` does the "Report" steps of the Continuous Curation pipeline.

```
report:
  runs-on: ubuntu-22.04
  # 'needs: test' makes this job wait for the 'test' job to finish.
  needs: test
  # 'if: always()' ensures this job runs even if the 'test' job failed.
  if: always()
  steps:
    # Checks-out your repository under $GITHUB_WORKSPACE, so your job can access it
    - uses: actions/checkout@v3
      with:
        ref: ${{ github.head_ref || github.ref_name }}
        fetch-depth: 0

    # Install everything I need
    - name: Install Dependencies
      run: |
        python -m pip install --upgrade pip
        python -m pip install -r requirements.txt

    # Generate the analysis reports
    - name: Generate Analysis Reports
      run: |
        python code/scripts/generate_growth_report.py
        python code/scripts/generate_pathway_report.py
        python code/scripts/generate_biomass_table.py
```

```
      python code/scripts/check_cue_values.py

  - name: Mention PR# in README.md
    env:
      PR_NUMBER: ${{ github.event.number }}
    run: sed -i -e "s/[[:digit:]]\{3,4\}\*\* (TEST-AND-REPORT)/$PR_NUMBER\*\* (TEST-AND-REPORT)/" c

  # You cannot know your PR number before opening the PR, so `--pr` is
  # optional when deprecating something. Fill in the blanks here, the same
  # way the step above stamps the PR number into code/scripts/results/README.md.
  # Only empty cells are touched, so rows attributed to a different PR or
  # to an issue are left alone.
  - name: Stamp PR# into deprecated identifier lists
    env:
      PR_NUMBER: ${{ github.event.number }}
    run: PYTHONPATH=code python -m tools.deprecate stamp-pr "$PR_NUMBER"

  # Auto-commit the changes directly without stashing
  - name: Auto-commit results
    uses: stefanzweifel/git-auto-commit-action@v4
    with:
      commit_user_name: action-bot
      commit_message: "chore: add Test-and-Report results"
      file_pattern: code/scripts/results/* data/deprecated_identifiers/*.tsv
    env:
      GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
      PR_NUMBER: ${{ github.event.number }}

  - name: Get commit SHA
    id: get-commit-sha
    run: echo "COMMIT_SHA=$(git rev-parse HEAD)" >> $GITHUB_ENV

  - name: Post comment
    uses: NejcZdovc/comment-pr@v2
    with:
      file: "ci_comment.md"
      identifier: "GITHUB_COMMENT_TEST_AND_REPORT"
    env:
      GITHUB_TOKEN: ${{secrets.GITHUB_TOKEN}}
      GH_ACTION_RUN: ${{github.run_id}}
      COMMIT_SHA: ${{ env.COMMIT_SHA }}
```

4.4 Step 4) Release

- What is semantic versioning
  - Semantic Versioning is a way to define your program's version based on the type of changes you've introduced. It's defined as a three-number string (separated with a period) in the format of MAJOR.MINOR.PATCH.
  - MAJOR version when you make incompatible API changes
  - MINOR version when you add functionality in a backward compatible manner
  - PATCH version when you make backward compatible bug fixes
- What counts as a major/minor/patch change for a GEM
  - major: an ID or file layout breaks, or any growth call flips
  - minor: the model file changes without flipping a call

    - patch: the model file is unchanged
- What counts as a new version
    - With continuous curation (as in continuous integration) the `develop` branch is always tested (there shouldn't be anything broken on the `develop` branch), so a release isn't any better quality
    - A release is really just a fixed point that someone can point at and cite
    - Possible triggers of when to make a release
        - When the model leaves the lab
            - i.e., when you publish a paper, give a talk, or send the model to a collaborator for them to use
            - So that no one outside the lab/curators is ever pointing to the `develop` branch
        - When the model needs a major bump
            - Whenever you have a growth call flipped or a broken ID
        - Time-based
            - e.g. every week, or every month
            - Keeps changes from piling up, and from `develop` getting too far ahead of `main`, but some of the changes may be so small (patches), that they are meaningless on their own
        - Every merged model change (continuous delivery)
            - Every change to the model (i.e., every merge to `develop`) gets its own release
            - now every version is a very small change
            - This is continuous delivery in software engineering, and since releases are automated, there is no real cost to doing this, but most of the releases aren't really useful
    - We sit somewhere between a & b, with multiple releases before publication as major changes in the model occurred
        - trigger a (publication) is the hard rule

1. Decide what size bump is required
    - Run locally: `PYTHONPATH=code python -m tools.release check`
2. On GitHub -> Actions -> Prepare-Release -> Run Workflow
    - Branch: develop
    - Kind: whatever size bump you decided on in 1
3. Prepare-Release runs
    - It commits changes to develop with the message `release: {new version number}`
        - Updates version.txt
        - Updates the CHANGELOG.md entry
    - And opens a PR from `develop` into `main` called `release: {new version number}`
4. Upon the release PR being opened, Release-Checks runs
    - MEMOTE report (1)
    - MACAW (11)
5. Wait for Release-Checks to finish
    - Can take ~15 min because of MACAW
6. Then merge the PR into main
7. The merge triggers the Publish workflow to run
    - Exports model to all formats
    - Make a tag for the new release
    - Creates the GuitHub release
        - Zenodo integration makes the DOI
    - Deploys MEMOTE to Pages

4.5 Step 5) Run

- This is the fun part, where you, or others, actually try to use the model
- Who might be using the model
    - Yourself- as the person making the model- you probably have questions you are interested in and simulations you would like to run
    - Your collaborators

    - Strangers- people who read your paper and want to use your model- potentially years after initial publication
- Making the model available to run throughout the curation process is what makes our curation "continuous", we aren't trying to finish the model before every using it, but will update the model as issues are found in use
- This means that you can curate the model in order of scientific questions- curating the pathways relevant to specific questions first, rather than in an arbitrary order
    - This means that the model will be better in regions of the metabolic network of previous interest
- This helps discover unexpected problems in the model- tests can only catch expected problems
- For tutorials on how to use a GEM to run FBA, see COBRA tutorials
    - MATLAB: https://opencobra.github.io/cobratoolbox/stable/tutorials/
    - COBRApy: https://cobrapy.readthedocs.io/en/latest/

4.6 Step 6) Monitor

- Typically there is limited feedback after publication- someone might try to recreate your results or use your model, but if it fails, they have no where to turn
- GitHub issues can be used to plan, discuss, and track work
- anyone can open an issue
- Use issues to keep track of bugs, enhancements, or other requests
- When to open an issue
    - You find something wrong
    - Bug/weird simulation results
    - New data to add
    - Missing feature you would like the model to have
    - Documentation confusing or unclear
    - Any type of feedback
- Open an issue
    - How to open an issue
        1. Go to the main page of the repository
        2. Click Issues
        3. Click New Issue
    - GitHub issues are written in GitHub Flavored Markdown
- What belongs in an issue
    - A concise and descriptive title
        - e.g. model does not grow on glucose, not model wrong, or "I tried to simulate growth on glucose, and got 0.00"
    - In the description
        - Detailed description
        - Provide context- including what you expected to happen versus what actually occurred
        - Step-by-step instructions for replicating the bug
            - include code, logs, or screenshots
        - Environment details
            - What OS are you using, what version of tools (e.g., COBRApy) are you using, what version of the model are you using, did you start from a release, from the main branch, from the develop branch
    - Note your issue is public, so don't include anything you don't want shared
    - Other repos (Human-GEM) have templates for reporting bugs, which can be helpful to ensure that someone gives all of the information needed to recreate and investigate the bug
- Anyone can comment on issues
- Unlike in traditional software this is a more human-dependent step- we aren't just getting crash reports, we require people to really run the model and manually open issues
- While this might seem trivial, this is a critical step, because without opening issues curation becomes a straight line, not a loop, and can't be continuous any more- so all issues are welcome!

5 Why can't you just make it for me?

- Can't you just make an installable tool that makes all of this for me?
- It is very possible to make a template- i.e. Standard-GEM
- And very possible to have packages with importable tests and shared functions
- Everyone will have different data, every model has it's own quirks, its worth writing your own to really get full coverage of your model

6 Develop Good Code/GitHub Habits

This is a summary of established best practices, not new to us (13) * If it isn't pushed, it doesn't exist * Commit often, push daily * One commit- one reasons * Work on a branch * Lets you be braver, you never have to worry about "breaking" anything * Don't let your branch live too long * If you've put a date or _final on a version on GitHub, you've done something wrong * i.e. never have model_v3_final_FINAL.xml * Write down the why, the diff already records the what * Use issues like a lab notebook * Future you will NOT remember the why * Don't copy and paste- if you are using the same code more than once it should be a function * That way if there's something wring, they are wrong in the same way, and they are never slightly different * Make a function with arguments, don't copy and tweak code * And once you make a function, make a test * No magic numbers * Use named constants- define them once and use them across scripts * e.g., `GROWTH_THRESHOLD` * If you're doing something by hand more than twice, automate it * Never edit data in place * I.e., don't download an excel file, and add a new column and save it, now you've lost the original. Instead, keep the orginal, and in a new script, read it, make the changes, and save the copy to a new file * Assert, don't eyeball * Don't store the same fact twice * Things will drift * If you much, have a test to ensure that they don't drift * e.g., my removed reactions list

7 Glossary

- **Artifact**:
  - *In software engineering*:
  - *In continuous curation*:
- **Branch**: A branch is a parallel version of a repository. It is contained within the repository, but does not affect the primary or main branch allowing you to work freely without disrupting the "live" version. [1].
- **Commit**: A commit, or "revision", is an individual change to a file (or set of files). When you make a commit to save your work, Git creates a unique ID (a.k.a. the "SHA" or "hash") that allows you to keep record of the specific changes committed along with who made them and when. Commits usually contain a commit message which is a brief description of what changes were made. [2].
- **Commit Message**: Short, descriptive text that accompanies a commit and communicates the change the commit is introducing [3].
- **Conflict**: A situation where two branches have changes in a file that Git cannot automatically merge, requiring manual resolution [4].
- **Continuous Integration (CI)**: A development practice where team members frequently integrate their code into a shared repository, often multiple times a day. Each integration is verified by automated builds and tests to detect errors early [5].
- **Continuous Delivery**: A software development practice where teams keep their product in a deployable state at all times, but deployment still requires a manual decision [6].

[1] https://docs.github.com/en/get-started/learning-about-github/github-glossary
[2] https://docs.github.com/en/get-started/learning-about-github/github-glossary
[3] https://docs.github.com/en/get-started/learning-about-github/github-glossary
[4] https://informatics.fas.harvard.edu/resources/glossary/#git-terms
[5] https://www.agile-academy.com/en/agile-dictionary/
[6] https://www.agile-academy.com/en/agile-dictionary/

- **Continuous Deployment**: A software development practice where every change that passes all automated tests is released to production automatically [7].
- **Curation**:
  - *In continuous curation*:
- **Diff**: A diff is the difference in changes between two commits, or saved changes. The diff will visually describe what was added or removed from a file since its last commit [8].
- **Feature Branch**: A branch used to experiment with a new feature or fix an issue that is not in production. Also called a topic branch [9].
- **Git**: Git is an open source program for tracking changes in text files [10].
- **GitFlow**:
- **GitHub**: A web-based platform that facilitates Git's use for collaboration between individuals. Other web-based platforms include GitLab and BitBucket [11].
- **GitHub Actions**:
- **Issue**: Issues are suggested improvements, tasks or questions related to the repository. Issues can be created by anyone (for public repositories), and are moderated by repository collaborators. Each issue contains its own discussion thread. You can also categorize an issue with labels and assign it to someone [12].
- **JSON**:
- **Local**:
- **Merge**: Merging takes the changes from one branch (in the same repository or from a fork), and applies them into another. This often happens as a "pull request" (which can be thought of as a request to merge), or via the command line [13].
- **Monitor**:
  - *In software engineering*:
  - *In continuous curation*:
- **Pull**: The process of integrating changes from one version of a repository to another (e.g. from a fork back to the original repo, or from a branch back to the main branch). There are two general use cases: 1) When the owner of a repository makes changes to it, you pull those changes into your local copy. 2) When you make changes to a forked repository or a branch of a repository and want to incorporate the changes back to the original repo or branch, you initiate a pull request, and then whoever is in charge of the original repository can pull those changes in [14].
- **Pull Request**: Pull requests are proposed changes to a repository submitted by a user and accepted or rejected by a repository's collaborators [15].
- **Push**: To push means to send your committed changes to a remote repository on GitHub.com. For instance, if you change something locally, you can push those changes so that others may access them [16].
- **Release**:
  - *In software engineering*:
  - *In continuous curation*:
- **Remote Repository**: This is the version of a repository or branch that is hosted on a server, most likely GitHub.com [17].
- **Report**:
  - *In software engineering*:
  - *In continuous curation*:
- **Repository/Repo**: A repository is the most basic element of GitHub. They're easiest to imagine as a

[7] https://www.agile-academy.com/en/agile-dictionary/
[8] https://docs.github.com/en/get-started/learning-about-github/github-glossary
[9] https://docs.github.com/en/get-started/learning-about-github/github-glossary
[10] https://docs.github.com/en/get-started/learning-about-github/github-glossary
[11] https://informatics.fas.harvard.edu/resources/glossary/#git-terms
[12] https://docs.github.com/en/get-started/learning-about-github/github-glossary
[13] https://informatics.fas.harvard.edu/resources/glossary/#git-terms
[14] https://informatics.fas.harvard.edu/resources/glossary/#git-terms
[15] https://docs.github.com/en/get-started/learning-about-github/github-glossary
[16] https://docs.github.com/en/get-started/learning-about-github/github-glossary
[17] https://docs.github.com/en/get-started/learning-about-github/github-glossary

project's folder. A repository contains all of the project files (including documentation), and stores each file's revision history. Repositories can have multiple collaborators and can be either public or private [18].

- **Run**:
  - *In software engineering*:
  - *In continuous curation*:
- **Runner**:
- **SBML**:
- **Script**:
  - *In software engineering*:
  - *In continuous curation*:
- **Semantic Versioning**:
- **Test**:
  - *In software engineering*:
  - *In continuous curation*:
- **Trunk-Based Development**:
- **Version Control**: The process of tracking changes to files over time, allowing you to recall specific versions later [19].
- **XML**:

References

1. Lieven C, others. 2020. MEMOTE for standardized genome-scale metabolic model testing. Nature Biotechnology 38:272–276.

2. Anton M, others. 2023. standard-GEM: Standardization of open-source genome-scale metabolic models. bioRxiv https://doi.org/10.1101/2023.03.21.512712.

3. Meszaros G. 2007. xUnit test patterns: Refactoring test code. Addison-Wesley.

4. Martin RC. 2008. Clean code: A handbook of agile software craftsmanship. Prentice Hall.

5. Reed JL, Vo TD, Schilling CH, Palsson BO. 2003. An expanded genome-scale model of Escherichia coli K-12 (iJR904 GSM/GPR). Genome Biology 4:R54.

6. Driessen V. 2010. A successful Git branching model. https://nvie.com/posts/a-successful-git-branching-model/.

7. Hucka M, Bergmann FT, Chaouiya C, Dräger A, Hoops S, Keating SM, König M, Le Novère N, Myers CJ, Olivier BG, Sahle S, Schaff JC, Sheriff R, Smith LP, Waltemath D, Wilkinson DJ, Zhang F. 2019. The Systems Biology Markup Language (SBML): Language specification for Level 3 Version 2 Core Release 2. Journal of Integrative Bioinformatics 16:20190021.

8. Olivier BG, Bergmann FT. 2018. SBML Level 3 Package: Flux Balance Constraints version 2. Journal of Integrative Bioinformatics 15:20170082.

9. Thiele I, Palsson BØ. 2010. A protocol for generating a high-quality genome-scale metabolic reconstruction. Nature Protocols 5:93–121.

10. Kanewala U, Bieman JM. 2014. Testing scientific software: A systematic literature review. Information and Software Technology 56:1219–1232.

[18] https://docs.github.com/en/get-started/learning-about-github/github-glossary
[19] https://informatics.fas.harvard.edu/resources/glossary/#git-terms

11. Moyer D, others. 2025. MACAW: A method for semi-automatic detection of errors in genome-scale metabolic models. Genome Biology https://doi.org/10.1186/s13059-025-03533-6.

12. Taschuk M, Wilson G. 2017. Ten simple rules for making research software more robust. PLOS Computational Biology 13:e1005412.

13. Wilson G, Bryan J, Cranston K, Kitzes J, Nederbragt L, Teal TK. 2017. Good enough practices in scientific computing. PLOS Computational Biology 13:e1005510.